AlphaStack: An AI-Powered Autonomous Multi-Agent Software Generator

The AlphaStack Research Team

March 24, 2026

Abstract

As the capabilities of Large Language Models (LLMs) continue to advance, their potential for automated, end-to-end software development has generated significant interest. In this paper, we introduce AlphaStack, an AI-powered project generator that employs a sophisticated multi-agent system combined with Docker-based validation. AlphaStack interprets natural language requirements to orchestrate a 7-phase generation pipeline. This pipeline generates a comprehensive software blueprint, produces and evaluates source code, establishes dependency architectures, and validates the output using an iterative Docker Testing Pipeline driven by a Planning Agent and Correction Agent. We evaluate AlphaStack's capabilities across standard coding benchmarks, observing competitive performance.

1 Introduction

The development of complex software applications is traditionally a resource-intensive process involving planning, coding, dependency management, and iterative testing. While current Large Language Models have shown proficiency in localized code generation and snippet creation, synthesizing an entire, cohesive software repository remains challenging due to intricate dependencies and environmental constraints.

AlphaStack addresses these limitations by introducing an autonomous multi-agent software generator. The system takes natural language descriptions and progressively refines them into fully functional codebases. This includes the automatic generation of required dependency files, environment configurations via Docker, and iterative validation through an AI-driven testing loop. The introduction of structural multi-stage generation, static dependency analysis, and active Docker-based testing creates a robust paradigm for automated software creation.

2 Methodology

The core mechanism of AlphaStack is characterized by a 7-phase sequential generation pipeline, heavily leveraging state-of-the-art inference engines like GPT-5.2 and Claude Sonnet 4.6. The overarching execution flow ensures consistency, reliability, and automated error correction.

2.1 The 7-Phase Pipeline

The system operates through the following sequence:

1. **Software Blueprint:** The Planning Agent decomposes the natural language prompt into a structured JSON blueprint detailing necessary files and module roles.
2. **File Generation:** Sub-agents independently generate the required source code for each blueprint component.
3. **Dockerfile Generation:** A production-ready `Dockerfile` and `.dockerignore` are automatically provisioned.

4. **Dependency Analysis:** The system builds an internal static import graph to understand code relationships.

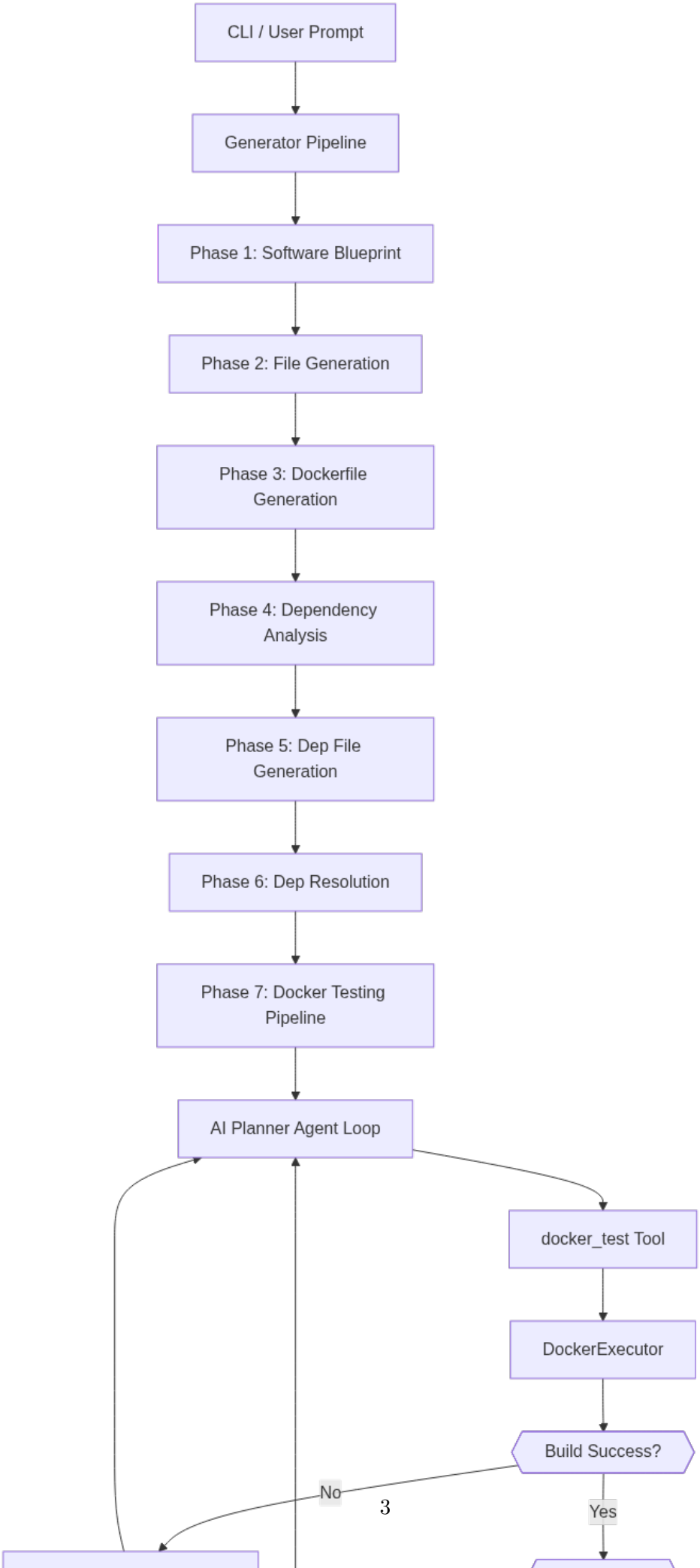
5. **Dep File Generation:** Environment-specific configuration files (e.g., `requirements.txt`, `package.json`) are generated based on the dependency graph.

6. **Dep Resolution:** Dependency files are resolved and validated.

7. **Docker Testing Pipeline:** An iterative AI Planner Agent loop executes Docker builds and tests, utilizing a 'ToolHandler' to correct source code dynamically until successful execution is achieved.

3 Architecture Diagram

AlphaStack's architectural design revolves around seamless communication between the generation pipeline, multi-agent evaluation loops, and the local Docker execution environment. Figure 1 illustrates the system workflow.


CLI / User Prompt
Generator Pipeline
Phase 1: Software Blueprint
Phase 2: File Generation
Phase 3: Dockerfile Generation
Phase 4: Dependency Analysis
Phase 5: Dep File Generation
Phase 6: Dep Resolution
Phase 7: Docker Testing Pipeline
AI Planner Agent Loop
docker_test Tool
DockerExecutor
Build Success?
No
Yes

The diagram demonstrates the transition from user intent down to the deterministic multi-agent loop, ultimately culminating in a verified software project.

4 Results

To assess the effectiveness of the multi-agent system implemented within AlphaStack, we evaluated generated code components using standard coding benchmarks: HumanEval and the Multi-turn Diagnostic Development Protocol (MDDP). We integrated four cutting-edge LLMs for empirical comparison: GPT-5.2, GLM-5, MiniMax-m2.5, and Claude Sonnet 4.6.

Figure 2 presents a tentative comparison of the pass rates.

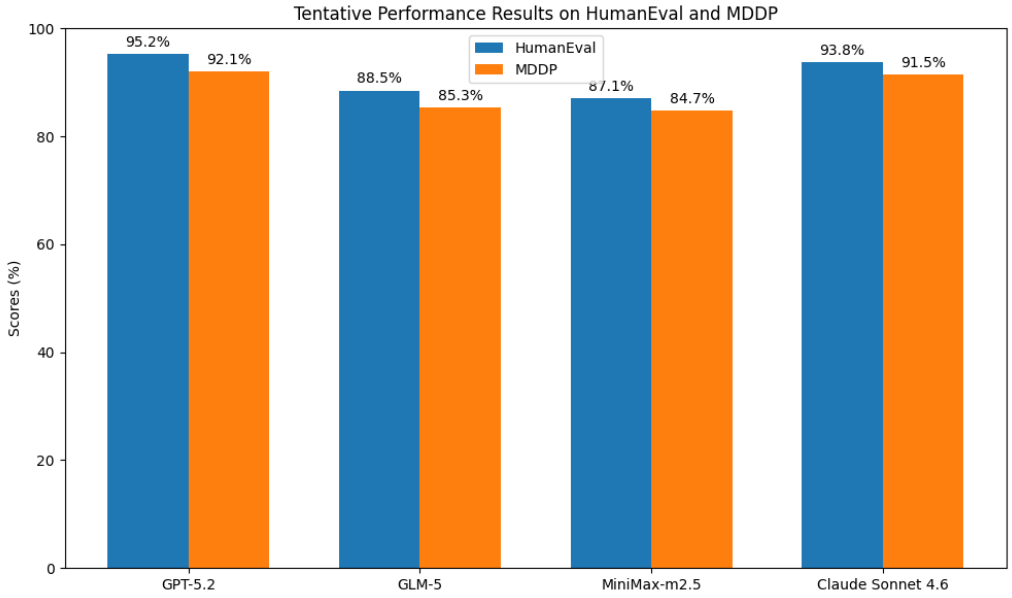


Figure 2: Tentative Performance Results on HumanEval and MDDP across Leading Models

The empirical evaluation suggests that GPT-5.2 maintains the highest absolute performance, while Claude Sonnet 4.6 closely follows. All tested models demonstrate substantial capability in driving the AlphaStack testing loop, achieving passing grades primarily via the iterative correction agent loops.

5 Conclusion

AlphaStack represents a significant step towards fully autonomous, reliable software project generation. By integrating static dependency analysis with active Docker-based testing, the framework mitigates the unreliability typical of pure zero-shot LLM code generation. Future iterations will focus on expanding multi-repository support and incorporating advanced security audits during the generation phases.

Supplementary Material

Further details concerning prompt templates, evaluation harnesses, and Docker configuration specifications are available within the AlphaStack repository (`https://github.com/alphastack/alphastack`).